

Machine Learning

Lecture 4

K-Nearest Neighbor Classifiers and Hyperparameter Optimization

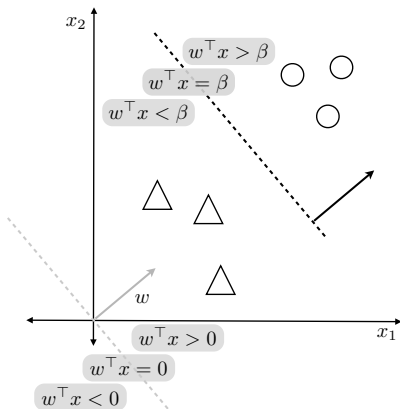
Felix Bießmann

Berliner Hochschule für Technik & Einstein Center for Digital Future

Overview of today's lecture

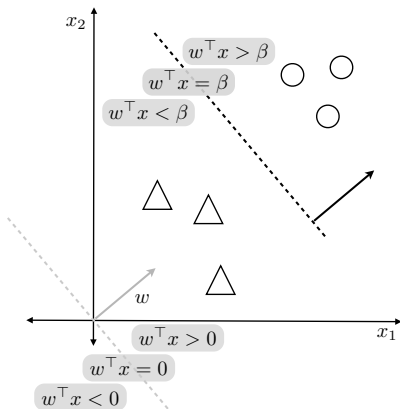
- Last week: *Nearest Centroid Classifier* (NCC)
- NCCs are *linear classifiers*
- NCCs cannot find non-linear decision boundaries
- This lecture introduces a simple but *non-linear* classifier:
- K-Nearest Neighbor Classifier (KNN)
 - KNN is very simple to implement
 - KNN can be really hard to beat

Linear Classification



- What is a good w ?
- Some proposals:
 - Logistic Regression
 - Perceptrons
 - Support Vector Machines
 - Ridge Regression
- Linear methods have different ways of defining what a good w is

Linear Classification



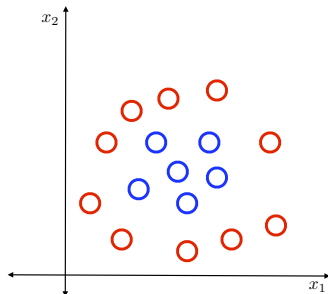
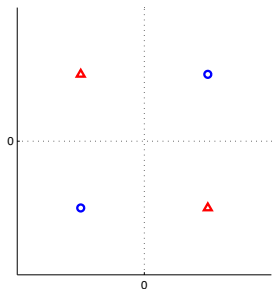
Nearest Centroid Classification is a simple linear classifier

$$w = \mu_o - \mu_{\Delta} \quad (1)$$

$$\beta = -1/2(\mu_o^\top \mu_o - \mu_{\Delta}^\top \mu_{\Delta})$$

Problems with Linear Classification

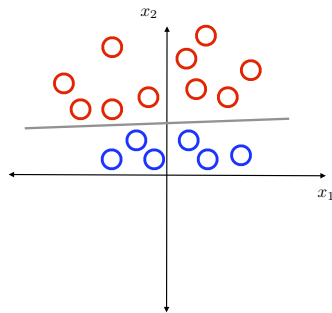
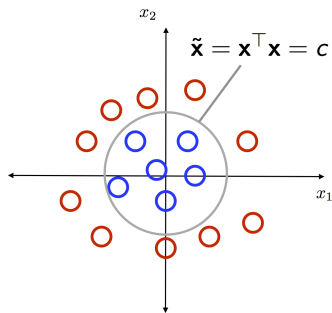
Linear Classifiers fail on non-linear problems:



Try to separate these two-class data sets with **a single line**

Problems with Linear Classification

If we can model the non-linearity,
we can create new features $\tilde{\mathbf{x}}$ which are linearly separable



But what if we do not know the non-linearity?

K-Nearest Neighbor Classifier

- Nearest Centroid Classifiers require estimation of centroids
- K-Nearest Neighbor is simpler
- Idea:
 1. Find the k closest neighbors for a new data point $\mathbf{x}$
 2. Look up labels of k closest neighbors
 3. Assign majority vote label for $\mathbf{x}$

K-Nearest Neighbor Classifier

We do not need to train a model for KNN –
the data is the model

But (as with NCC) we need a distance function

Usually the euclidean distance is chosen:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_2 \quad j \in [1, \dots, N] \quad (2)$$

where N is the number of data points

K-Nearest Neighbor Classifier: Pseudocode

Algorithm 1 K-Nearest Neighbour Prediction

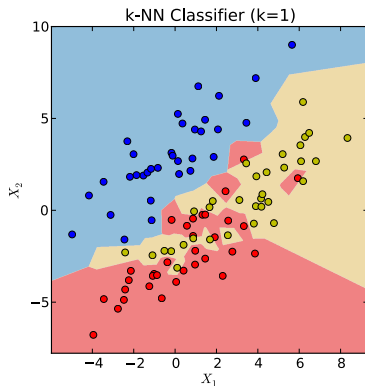
Require: Test data point $\mathbf{x}_i \in \mathbb{R}^D$, training data $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_N] \in \mathbb{R}^{D \times N}$, corresponding labels $\mathbf{y} = [y_1, \dots, y_N]^T \in \{-1, +1\}^N$, number of neighbours K

Ensure: Predicted label $\mathbf{y}_i$

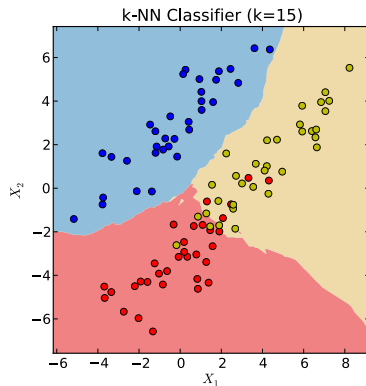
- 1: **for** $\mathbf{x}_j$ in $\mathbf{X}$ **do**
 - 2: # Compute distance between test data $\mathbf{x}_i$ and training data $\mathbf{x}_j$
 - 3: **end for**
 - 4: Find most frequent label of the k -nearest neighbors
 - 5: Predicted label $\mathbf{y}_i$ is that with most votes (ties are broken at random)
-

K-Nearest Neighbor Classifier

Toy data problem: Linear classification



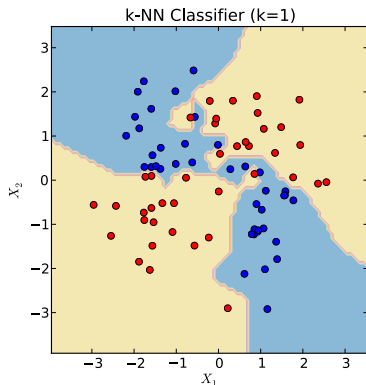
$k = 1$



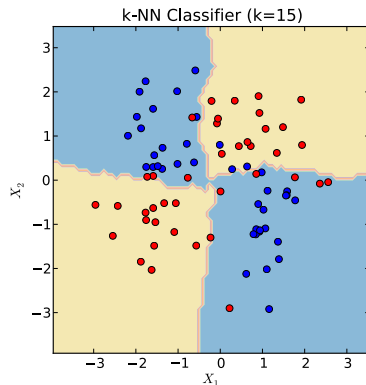
$k = 15$

K-Nearest Neighbor Classifier

Toy data problem: Nonlinear classification



$k = 1$



$k = 15$

Problems with KNN

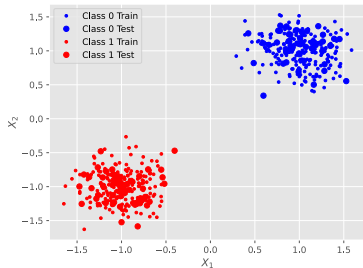
- Hyperparameter k needs to be set appropriately:
 - k small: complex decision boundaries
 - k large: smooth/simple decision boundaries
- Consider N data points $\mathbf{x} \in \mathbb{R}^D$
- Find K Neighbors requires $O(NND)$ operations
- For large data sets this is too costly
- Speedups can be gained by:
 - Trees for distance computations
 - Locality Sensitive Hashing for finding neighbors

Overfitting

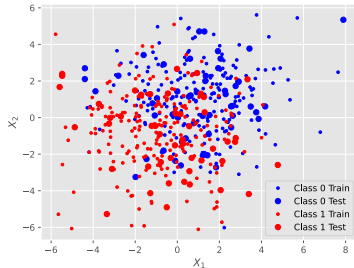
How can we set k such that KNN has the right complexity for generalizing to new data?

Overfitting

No noise (not realistic)



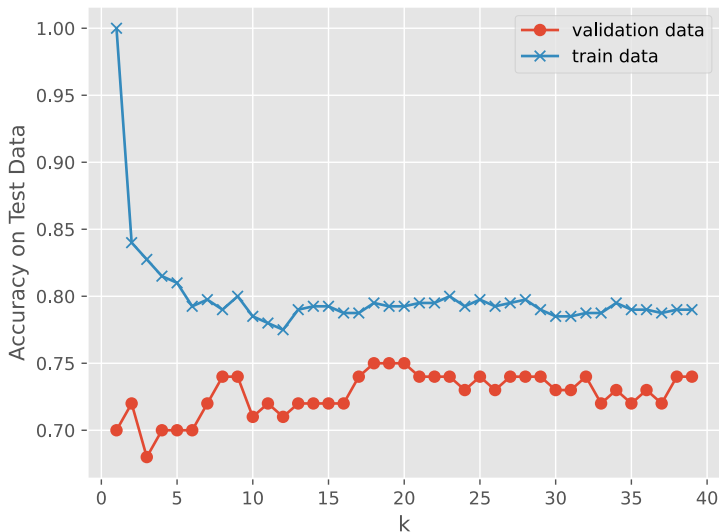
With noise (realistic)



How will affect the choice of k
our classification on train / test data?

Overfitting

Linear Classification



Overfitting

- Hyperparameter k controls complexity of KNN predictions
 - Often data is complex because it's noisy
- k too small leads to **overfitting**
- ! We don't want to predict noise
 - ! Decreasing k will *always improve* accuracy on training data
- How can we learn k such that KNN has the right complexity for generalizing to new data?

Model Selection

- k can be learned by **Hyperparameter Optimization**
- Several options
 - Grid Search
 - Random Search
 - Bayesian Global Optimization

Grid Search with Cross-Validation

- Define a **grid** of hyperparameter candidates $k \in 1, 2, 3, \dots$
- Split data set in F different **training** and **test** data folds

$$\left[\underbrace{x_1, x_2, x_3, x_4}_{\mathcal{F}_1^{\text{train}}}, \underbrace{x_5, x_6}_{\mathcal{F}_1^{\text{test}}} \right]$$

- For each k in grid:
 - Train** your model on the training data on $\mathcal{F}^{\text{train}}$
 - Test** your model on the test data on $\mathcal{F}^{\text{test}}$
- Then choose the k that worked best on the held-out test data.

Summary

K-Nearest Neighbor (KNN) Classifier

- Simple nonlinear classifier

- State-of-the-art prediction performance

- No training required

- Needs to evaluate pairwise distances of **all** data points

→ **Slow**